

---

# Inductive Bias Meta-Learning with Generative Models

---

Name Surname  
MIPT  
Moscow, Russia  
email@phystech.edu

Name Surname  
MIPT  
Moscow, Russia  
email@phystech.edu

## Abstract

This paper investigates inductive bias in machine learning models. By inductive bias we mean the preference of a model for certain types of functions or data structures over others. To analyze the inductive bias of a fixed model, we consider a problem of finding data that this model can fit and generalize on particularly well. Previous work demonstrated that generating labels for a fixed dataset allows one to extract the inductive bias. Here, we extend this approach by proposing a method for generating full synthetic datasets. We train a generative model to produce datasets on which the target model achieves strong generalization performance. We test the proposed framework on CNNs and RNNs, and analyze obtained datasets for each model.

**Keywords:** inductive bias, meta-learning, generative models.

## 1 Introduction

Inductive bias plays a fundamental role in a model’s ability to generalize [1]. Without additional assumptions, any hypothesis that fits a finite training dataset is considered equally likely by the model, which can lead to poor generalization. Thus, effective generalization requires a set of prior assumptions about the data, and these assumptions constitute the model’s inductive bias. Understanding and exploiting inductive bias can be a powerful tool for enhancing model performance.

The term “inductive bias” does not have a single, strict definition. For instance, inductive bias can mean underlying assumptions about the domain and task or specific implementation choices that constrain or order the space of hypotheses [2]. In this paper, by inductive bias we mean a model’s preference for certain types of functions or data structures over others, expressed through better generalization performance. For example, convolutional neural networks (CNNs) are inductively biased towards data with local structure and translation invariance [3–5], while recurrent neural networks (RNNs) are inductively biased towards ordered data with sequential structure [6–8].

A clear understanding of the correspondence between models (or their specific components) and inductive biases is useful for solving machine learning problems. However, this task is highly non-trivial: for most models with complex architecture, the problem of detecting inductive bias is analytically intractable. Several approaches have been developed to extract and characterize the inductive bias of models. Boopathy et al. [9] proposed a method to quantify the amount of inductive bias of tasks or models as the amount of information required to specify well-generalizing models within a given hypothesis space. Si et al. [10] investigated the inductive bias of in-context learning by constructing underspecified demonstrations. Li et al. [11] proposed a framework to capture the inductive biases in a learning system by meta-learning Gaussian process kernel hyperparameters from its predictions.

One recent result in this area is the meta-learning framework introduced by Dorrell et al. [12], which identifies the inductive bias of differentiable supervised learning algorithms by generating labels for a

fixed dataset. For all its strengths, this approach has certain limitations. First, it requires access to input data or its distribution. Second, its analysis of inductive bias is constrained by the fixed dataset and its structure.

Here, we propose to extend this approach by generating entire synthetic datasets. The meta-learning loop proceeds as follows: an outer generative model, or meta-learner, produces a dataset, which is used to train and test an inner model, or learner, whose inductive bias we aim to extract. After the learner is trained, the quality of its predictions on test data from this dataset is incorporated into the meta-learner’s loss function. Thus, the meta-learner learns to generate datasets on which the target model shows the best generalization performance; these are datasets towards which it is inductively biased.

The proposed approach requires no knowledge of the input data distribution, making it more flexible. We expect it to enable a more comprehensive analysis of inductive bias.

**Contributions.** Our contributions can be summarized as follows:

- We present a generative meta-learning framework for extracting the inductive bias of a target model. Unlike prior work that generates only labels for samples from a fixed distribution, our approach produces entire synthetic datasets via a meta-learned generative model.
- We empirically evaluate our algorithm on CNNs and RNNs and compare the results with the known inductive biases of these architectures. For CNNs, we expect to detect a local structure in the generated datasets; for RNNs, we expect them to have a sequential structure.
- We highlight the implications of our findings for understanding and leveraging inductive bias. Our method is general-purpose: it can be used to extract inductive bias of any differentiable supervised learning algorithm and, consequently, to improve the generalization performance of models in machine learning problems.

**Outline.** The rest of the paper is organized as follows...

## 2 Problem statement

We study the problem of extracting and characterizing the inductive bias of learning models by finding the datasets that they generalize best. We now present a formal statement of the problem.

Let  $\mathbb{X} \subset \mathbb{R}^n$  be the input space,  $\mathbb{Y} \subset \mathbb{R}^m$  be the output space and  $\mathbf{f}_\theta : \mathbb{X} \rightarrow \mathbb{Y}$  be the target model, where  $\theta \in \Theta \subset \mathbb{R}^k$  are its parameters. Let  $\ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y})$  be the loss function of the target model.

Consider a parametric family of distributions  $p(\mathbf{x}|\mathbf{y}, \mathbf{w})$ , where  $\mathbf{w} \in \mathbb{W} \subset \mathbb{R}^l$  are the parameters of the distribution. For a fixed label  $\mathbf{y} \in \mathbb{Y}$ , a sample  $\mathbf{x} \sim p(\mathbf{x}|\mathbf{y}, \mathbf{w})$  is produced by the generator  $\mathbf{g}(\mathbf{w})$ .

We aim to find data distribution function  $p^*$ , corresponding to parameters  $\mathbf{w}^*$ , the solution to the following optimization problem:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{W}} \mathbb{E}_{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}} \sim p(\mathbf{x}|\mathbf{y}, \mathbf{w})} [\mathcal{L}(\mathbf{f}_\theta, \mathcal{D}_{\text{test}})] \quad \text{s.t.} \quad |\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}| = d$$

where

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{train}}} \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y}).$$

Function  $\mathcal{L}$  measures a model’s ability to generalize by calculating the prediction error on a test set and also contains regularization terms to avoid trivial solutions. For example, prediction error can be estimated as cross-entropy or mean squared error (MSE), defined respectively as:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{test}}} \sum_{i=1}^m y_i \log ([\mathbf{f}_{\hat{\theta}}(\mathbf{x})]_i), \quad \mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{test}}} \|\mathbf{y} - \mathbf{f}_{\hat{\theta}}(\mathbf{x})\|^2.$$

The regularization term can be the entropy of the found distribution, defined as:

$$\mathcal{R}_{\text{ent}}(\mathbf{w}) = -\mathbb{E}_{\mathbf{y}} \left[ \int_{\mathbb{X}} p(\mathbf{x}|\mathbf{y}, \mathbf{w}) \log p(\mathbf{x}|\mathbf{y}, \mathbf{w}) d\mathbf{x} \right].$$

### 3 Method

## 4 Experiments

### 4.1 Baseline experiment

First, we replicate the experiment by Dorell et al. [12] on extracting inductive bias via label generation. Their algorithm proceeds as follows: the meta-learner generates labels for a dataset, which is subsequently used to train the learner; the learner’s generalization performance on these labels is then evaluated and incorporated into the meta-loss function. In this way, the meta-learner is trained to find functions that the learner can easily generalize.

As the target model, we use linear regression, and as the meta-learner, a 4-layer MLP. The meta-loss function consists of three terms: the MSE of the linear regression on the test set, the Sinkhorn distance of the label distribution from a uniform distribution, and terms that account for previously obtained functions to ensure orthogonality with them.

As a result of the experiment, we obtained three orthogonal linear functions, as expected.

### 4.2 Proposed Framework Evaluation

To empirically validate our algorithm, we conducted computational experiments to extract the inductive bias of CNNs and RNNs. Specifically, we verify that the generative model learns to create datasets on which the target model shows the best generalization performance, investigate the influence of different regularization terms on the generated data, and attempt to interpret the obtained results in light of the well-known inductive biases of these architectures.

Data in our experiments is fully synthetic. For CNNs we generate datasets of images with a given number of classes and number of objects per class. As a generative model, we used a variational autoencoder (VAE) decoder. It transforms a vector from a latent space, consisting of class embedding and random noise sampled from a standard normal distribution  $\mathcal{N}(0, 1)$ , into a grayscale image.

The meta-learning process is implemented using the learn2learn library [13]. Specifically, to enable end-to-end differentiability, we wrap the inner learner in a MAML (Model-Agnostic Meta-Learning) [14] wrapper. This allows us to clone the learner for each meta-step, train it on the generated dataset, and still keep the computational graph needed to propagate gradients from the meta-loss back to the generator. We use Adam [15] as the optimizer for meta-learning.

As measures of generalization performance, we use MSE and cross-entropy and then compare them. We also conduct a comparative analysis of different types of regularization terms: the entropy of the distribution, the average distance between objects of the same class, and the singular values of the generator’s weight matrix.

List of expected figures:

- graph comparing different types of regulation (x-axis - meta-iterations, y-axis - accuracy, %);
- figure with examples of the resulting datasets;
- meta-loss change chart (x-axis - meta-iterations, y-axis - meta-loss).

### 4.3 Preliminary report

The figure 1 presents a comparative analysis of different regularization terms in experiments with CNNs. The blue curve corresponds to the experiment without regularization, the red curve represents regularization via the average distance between objects of the same class, and the green curve corresponds to regularization by penalizing small singular values of the generator’s weight matrices. As can be seen, without regularization, 100% accuracy is achieved faster (at about 230 iterations) due to the dataset degenerating into a trivial solution where all images within the same class are identical.

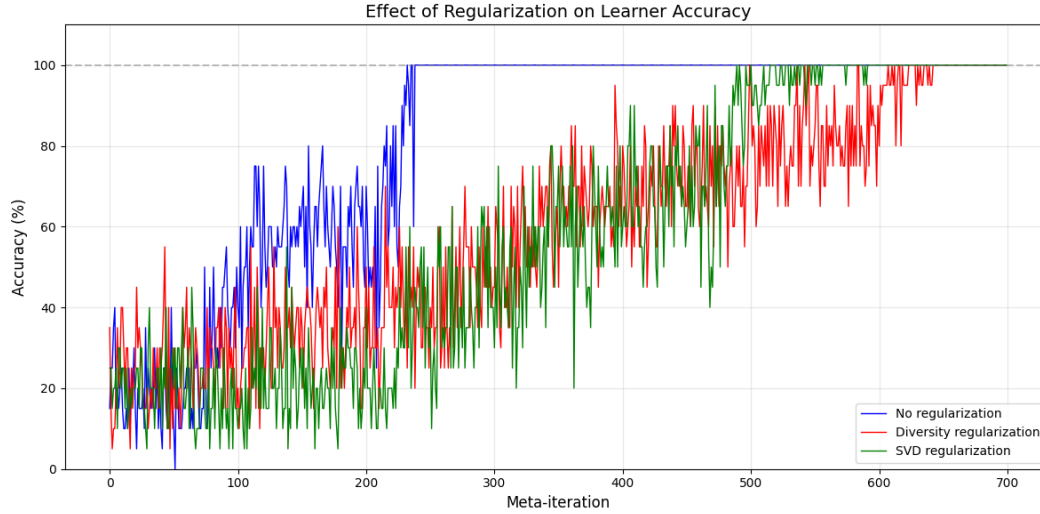


Figure 1: Comparison of different types of regularization terms for CNNs.

## 5 Discussion

## 6 Conclusion

## References

- [1] Tom M Mitchell. The need for biases in learning generalizations. 1980.
- [2] Foster John Provost and Bruce G Buchanan. Inductive policy: The pragmatics of bias selection. *Machine Learning*, 20(1):35–61, 1995.
- [3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [4] Zihao Wang and Lei Wu. Theoretical analysis of inductive biases in deep convolutional networks, 2024. URL <https://arxiv.org/abs/2305.08404>.
- [5] Nadav Cohen and Amnon Shashua. Inductive bias of deep convolutional networks through pooling geometry, 2017. URL <https://arxiv.org/abs/1605.06743>.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [7] Taiga Ishii, Ryo Ueda, and Yusuke Miyao. Empirical analysis of the inductive bias of recurrent neural networks by discrete fourier transform of output sequences, 2023. URL <https://arxiv.org/abs/2305.09178>.
- [8] Igor Dubinin and Felix Effenberger. Fading memory as inductive bias in residual recurrent networks. *Neural Networks*, 173:106179, 2024. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2024.106179>. URL <https://www.sciencedirect.com/science/article/pii/S0893608024001035>.
- [9] Akhilan Boopathy, William Yue, Jaedong Hwang, Abhiram Iyer, and Ila Fiete. Towards exact computation of inductive bias, 2024. URL <https://arxiv.org/abs/2406.15941>.
- [10] Chenglei Si, Dan Friedman, Nitish Joshi, Shi Feng, Danqi Chen, and He He. Measuring inductive biases of in-context learning with underspecified demonstrations, 2023. URL <https://arxiv.org/abs/2305.13299>.

- [11] Michael Y Li, Erin Grant, and Thomas L Griffiths. Meta-learning inductive biases of learning systems with gaussian processes. In *Fifth Workshop on Meta-Learning at the Conference on Neural Information Processing Systems*, 2021.
- [12] William Dorrell, Maria Yuffa, and Peter Latham. Meta-learning the inductive biases of simple neural circuits, 2023. URL <https://arxiv.org/abs/2211.13544>.
- [13] Sébastien M. R. Arnold, Praateek Mahajan, Debajyoti Datta, Ian Bunner, and Konstantinos Saitas Zarkias. learn2learn: A library for meta-learning research, 2020. URL <https://arxiv.org/abs/2008.12284>.
- [14] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017. URL <https://arxiv.org/abs/1703.03400>.
- [15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.

## **A Appendix / supplemental material**

### **A.1 Additional experiments / Proofs of Theorems**

TODO